
The Free Flip. Reverse Capability in Obfuscation Fine-Tuning Comes from Data Direction, Not a Contrastive Objective

Anonymous Author(s)

Abstract

Code obfuscation rewrites a program so that it still computes the same thing but is much harder to read. A model can be trained to *apply* such a rewrite, the forward direction, or to *undo* one, the reverse direction. Nikiema et al. [2025] report that fine-tuning on the forward direction destroys the reverse direction, a failure they name *cognitive specialization*, and propose Contrastive Fine-Tuning as a remedy that recovers 39–52% of reverse performance without reverse training data. The auxiliary instances that objective requires are themselves new data, so we test the two factors separately. In a 2×2 experiment on Qwen2.5-Coder-1.5B the objective is worth -0.2 percentage points (95% confidence interval $[-0.7, +0.3]$) while reversing the data is worth $+30.9$ points $[+29.3, +32.6]$. Reverse examples cost nothing to obtain, because every training pair read backwards is one, and at 7B an arm that replaces half the forward pairs with their reverses takes reverse success from 0.0% to 32.8% at identical training cost. Our reimplementations of the published method reaches 0.1%, below the untouched model’s 12.9%. We differ from the original in language, corpus size and one transform, so we claim non-reproduction on our corpus rather than in general. The recovered capability is narrower than reported, being at or below 2.3% on all three renaming transforms for every arm we trained, including reverse-only training. The 6 to 7 point drop we measure on a general coding benchmark is also not caused by the reverse data, since an arm trained on reverse data alone reaches the same reverse success and shows no drop. The general pattern we would draw attention to is that fine-tuning on one direction of a reversible transformation removes the other direction, and does so selectively, leaving general coding ability intact at 7B while reverse success falls from 12.9% to 0.0%. The missing direction is then unusually cheap to restore, because its training data is the data already in hand read backwards.

1 Introduction

Directional training and the direction it removes. Many useful code transformations are reversible in principle. Obfuscating a program rewrites it so that it behaves identically but is hard to read, by renaming `total_price` to `v_a3f2` or by replacing a function’s control flow with a dispatch loop. Each such rewrite has a before and an after, so a model can be asked to work in either of two directions (Figure 1). In the **forward** direction it is given readable code and must produce the obfuscated version. In the **reverse** direction it is given obfuscated code and must recover something readable and equivalent, which is the direction practitioners actually want.

The phenomenon this paper is about is that fine-tuning on one of those directions removes the other. At 7B, forward-only fine-tuning takes reverse success from 12.9% to 0.0%. The loss is also se-

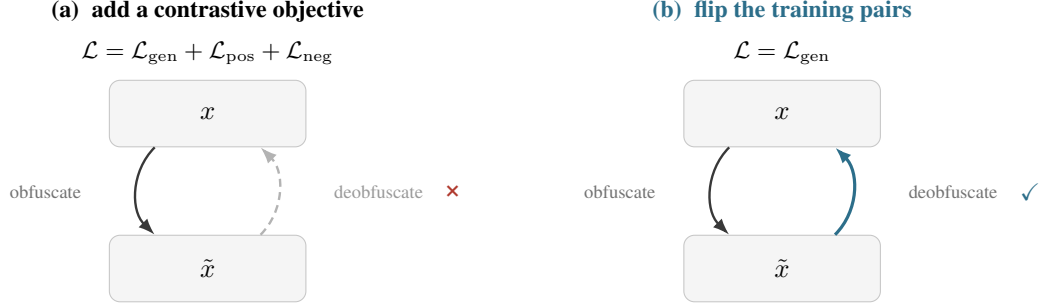


Figure 1: The attribution this paper tests. Both panels fine-tune the same model on the same obfuscation pairs. **(a)** Contrastive Fine-Tuning augments the generation loss with two equivalence-judgement terms and is credited with restoring the reverse direction. In our experiments it does not. Reverse success stays at zero and falls below the untouched model. **(b)** Keeping the plain generation loss and reading half of the same pairs backwards restores the reverse direction at identical training cost. The recovery comes from the direction of the data, not from the objective.

lective rather than a general degradation, which is what distinguishes it from ordinary catastrophic forgetting. The same model keeps its general coding ability completely intact, scoring .817 on HumanEval+ against .805 for the untouched model, so what fine-tuning removed was specifically the inverse of the mapping it was taught. A practitioner who fine-tunes a model to apply a transformation should expect to lose the ability to undo it, and to lose it without any warning sign in general benchmarks.

The reported remedy, and why it is an attribution problem. Nikiema et al. [2025] document this failure, name it *cognitive specialization*, and propose **Contrastive Fine-Tuning** (CFT) as a remedy. CFT adds two equivalence-judgement terms to the generation loss (Figure 1a), training the model to say whether two programs are semantically equivalent alongside learning to obfuscate. They report that this recovers 39–52% reverse success *without any reverse training data*, which is what makes the result striking, because it suggests the capability is unlocked by how the model is trained rather than by what it is shown.

That conclusion depends on an attribution that was not tested. CFT changes two things at once, adding an objective and adding the equivalence-judgement instances that objective requires, which are data the forward-only baseline never sees. A much cheaper candidate is available too. Every training example pairs a readable program with an obfuscated one, so reading a pair backwards is already a reverse example, obtained at no cost. If that alone restores the reverse direction, then the failure describes the data a recipe was given rather than the objective it was trained under. The original paper’s evaluation section does define a Bidirectional Fine-Tuning condition, forward generation plus reverse deobfuscation, but reports no result for it, so we run that comparison here.

What we find. The data direction accounts for essentially the entire effect and the objective for none of it (Figure 1b). A model trained on half forward and half reversed pairs, at identical cost to forward-only training, goes from 0.0% to 32.8% reverse success, while our reimplement of CFT reaches 0.1%. Two qualifications matter. The recovered capability is confined to rewrites that can in principle be undone, and training on both directions carries a general-capability cost that training on either one alone does not, so the cheapest correct fix depends on whether both directions are needed at deployment.

2 Setup

Programs and rewrites. We use Python functions that come with executable test inputs, drawn from APPS [Hendrycks et al., 2021], CRUXEval [Gu et al., 2024] and HumanEval [Chen et al., 2021]. Each function is rewritten under five obfuscating transforms. Three rewrite identifiers, by renaming them to misleading names (L1b), to random strings such as v_a3f2 (L1r), or to single letters with type annotations stripped (L2). Two rewrite structure, by flattening control flow into a dispatch loop (S1) or inserting always-true tests and unreachable code (S2). That split matters later, because only the structural rewrites can be undone. Full definitions are in Appendix G. Every

Table 1: The training mixtures compared in this paper. Only the mixture differs; the model, recipe and budget are otherwise identical. `mix50` replaces forward examples rather than adding to them, which is what makes it cost the same as `sft`.

arm	training examples	why it is in the experiment
<code>base</code>	none	the untouched control the original study omits
<code>sft</code>	forward only	the original study’s forward-only baseline
<code>cft</code>	forward + equivalence judgements	our reimplementation of the published method
<code>flip</code>	forward + the same pairs reversed	the declared but unreported baseline
<code>mix50</code>	half forward, half reversed	the decisive arm, cost-matched to <code>sft</code> with <i>less</i> supervision
<code>rev</code>	reverse only	how well reverse can be done at all
<code>fwd2x</code>	forward only, twice the epochs	rules out “it simply trained longer”
<code>cftflip</code>	judgements + both directions	the fourth cell of the 2×2

rewrite is applied to the original function on its own, never stacked, and is kept only if the rewritten program still produces the original outputs.

How we score a trial. For one trial let x be the original readable program, $\tilde{x}$ its obfuscated form, and $\hat{y}$ the model’s output. Four quantities build the criteria. $\text{parses}(\hat{y})$ is 1 when the output is non-empty and parses in the target language. $S(a, b) \in [0, 1]$ is CodeBLEU similarity between two programs [Ren et al., 2020]. $R(a) \in [0, 1]$ is readability, a weighted sum of identifier meaningfulness (0.55) with identifier length, line length and nesting depth (0.15 each), and it returns 0 for a non-parsing output, since the readability of a non-program is undefined and 0 is the only resolution that cannot manufacture a success. $E(\hat{y})$ is 1 when the output executes and reproduces x ’s outputs on every test input that ships with the program. Appendix F restates these as a table.

In the forward direction the model must emit an obfuscated program, and we score it by whether that program still computes the original outputs, so forward success is $E(\hat{y})$.

For the reverse direction we report two criteria, because the published one turns out to be permissive. The **published criterion** is the test used by Nikiema et al. [2025], which asks that the output be textually dissimilar to the obfuscated input and about as readable as the original,

$$\text{Pub}(\hat{y}) = \text{parses}(\hat{y}) \wedge [S(\hat{y}, \tilde{x}) < 0.4] \wedge [R(\hat{y}) \geq R(x) - 0.1], \quad (1)$$

where the thresholds are that paper’s values and the parses conjunct is ours. The guard is not optional, because without it the first inequality is satisfied by any reply that merely differs from the input, and a run of fixed placeholder strings scored 17–25% “reverse success” before we added it. It can only lower a reported number, never inflate one.

Strict success, which we use for every headline number, additionally requires that the recovered program run correctly,

$$\text{Strict}(\hat{y}) = \text{Pub}(\hat{y}) \wedge E(\hat{y}). \quad (2)$$

We report it because a deobfuscation that does not run has not recovered anything. We also track **echoing**, $\text{Echo}(\hat{y}) = \mathbf{1}[\hat{y} = \tilde{x}]$, which is how often the model simply returns its input, separately rather than letting such replies count towards accuracy.

One substitution matters for Eq. (1). The original study measures readability with the model of Scalabrino et al. [2018], which we do not have, so R is the proxy in Table 6. We checked what that costs by recomputing every result while varying the readability treatment, including dropping the term entirely, and strict success moves by at most 1.3 pp on any trained arm against an effect of 33 points (Appendix A). Execution and the similarity bound do essentially all the deciding.

What we compare. Every model in this paper is the same base model with a small set of adapter weights trained on top, using LoRA [Hu et al., 2022], and every one uses an identical recipe (rank 32, three epochs, learning rate 10^{-4} , batch 64, seed 17). The only thing that differs between them is the mixture of training examples, so no difference between them can come from hyperparameters. We call each mixture an **arm**, and Table 1 lists them.

Evaluation and uncertainty. We decode greedily on 300 held-out programs, using only programs that every transform succeeded on so that no two cells are scored on different populations. All intervals are 95% confidence intervals from a bootstrap that resamples *programs* rather than individual

Table 2: The 2×2 experiment. Qwen2.5-Coder-1.5B [Hui et al., 2024], strict reverse success in per cent, 300 programs and 18 000 graded attempts, all cells scored on the same programs. The right-hand panel decomposes the four cells into the effect of each factor.

	forward only	+ reversed pairs
no judgement objective	<code>sft</code> 0.3	<code>flip</code> 31.4
with judgement objective	<code>cft</code> 0.3	<code>cftflip</code> 31.1
effect of	estimate (pp)	95% CI
reversing the data	+30.9	[+29.3, +32.6]
the objective	-0.2	[-0.7, +0.3]
the two interacting	-0.3	[-1.3, +0.7]

test cases, because several test cases share a program and are therefore correlated. We write **pp** for percentage points. Greedy decoding is not bitwise repeatable here, since batching changes which token wins a near-tie, so two passes over the same programs differ on 6 to 8% of generations and move a headline rate by about 0.2 pp (Appendix A). Every table comes from a single pass, we compare arms only within a pass, and we never quote a cross-pass difference below 0.5 pp.

3 The objective contributes nothing; the data direction contributes everything

Table 2 varies the two factors CFT bundles together. One factor is whether the equivalence-judgement objective is present. The other is whether the training data contains the reversed pairs. All four cells use the same 300 programs and the same 18 000 graded attempts.

The objective’s contribution is not merely too small to detect. It is bounded under one percentage point in either direction, and that holds whether or not reversed data is present, with an interaction term that also spans zero. Adding the judgement objective to forward-only training moves reverse success by -0.1 pp $[-0.5, +0.3]$, and adding it on top of reversed data moves it by -0.3 pp $[-1.2, +0.5]$. Reversing the data, by contrast, moves it by roughly 31 points. Whatever recovers the reverse direction, it is not the objective.

Training longer does not explain it either. `fwd2x` trains forward-only for twice as many optimizer steps and reaches 0.6%, which is $+0.3$ pp over `sft` with an interval of $[-0.2, +0.8]$.

How much reversed data is needed, and it is much less than half. Appendix B replaces forward examples with reversed ones in increasing amounts. Replacing only 5% of them already recovers 26.1%, against 0.3% for forward-only, which is 85% of what a 50% share achieves. The remaining climb is real but small, worth $+4.5$ pp $[+3.2, +5.8]$, and it has flattened by the time a quarter of the data is reversed. So the intervention is not a matter of data volume. A small amount of exposure to the reverse direction captures most of the effect.

4 The effect is free, at 7B, on every axis of training cost

The obvious objection to Table 2 is that `flip` sees twice as many examples as `sft`. `mix50` answers it by *replacing* half the forward examples with reversed ones instead of adding them, so it trains on the same number of examples (7 383) for the same number of optimizer steps (346). Two quantities are worth separating here. **Sequence tokens** are all the tokens the model reads, which is what determines compute. **Supervised tokens** are the subset the training loss is actually computed on, which is what determines how much the model is taught. `mix50` uses $1.05\times$ the sequence tokens of `sft` and only $0.71\times$ the supervised tokens, so it is taught *less*, not more. Table 3 reports both directions at 7B.

`mix50` reaches $+32.8$ pp over `sft` $[+31.3, +34.4]$ while costing no more on any axis a practitioner pays for, and `flip`, which does spend $2.09\times$ the sequence tokens, buys only $+0.7$ pp more $[-0.3, +1.8]$, an interval spanning zero. The extra compute is not what produces the capability. The exposure is. The published method fares worst here, spending $2.65\times$ the compute of forward-only training to add 2% more supervised signal, because each equivalence judgement puts two whole programs in the prompt and supervises one word.

Table 3: Qwen2.5-Coder-7B, 300 programs, 21 000 graded attempts. *Forward* is scored by execution. *Reverse (strict)* requires the published criterion and that the output run correctly. *Echo* is how often the model returns its input unchanged.

arm	forward	reverse (strict)	reverse (published crit.)	echo
base (<i>untouched</i>)	.929	.129	.233	.073
sft (<i>forward only</i>)	.971	.000	.004	.182
fwd2x ($2 \times$ <i>epochs</i>)	.969	.001	.019	.071
cft (<i>published method</i>)	.975	.001	.002	.293
rev (<i>reverse only</i>)	.927	.329	.351	.000
flip	.971	.335	.353	.000
mix50 (<i>same cost as sft</i>)	.961	.328	.356	.000

Table 4: HumanEval+ pass@1 (164 problems, no scorer errors in any cell), grouped by whether the training mixture contains one direction of the rewrite or both. At 7B it is the grouping, not the amount of reverse data, that predicts the cost.

arm	1.5B	7B
base (<i>untouched</i>)	.646	.805
<i>trained on ONE direction</i>		
sft (<i>forward only</i>)	.329	.817
fwd2x (<i>forward</i> , $2 \times$ <i>epochs</i>)	n/a	.805
cft (<i>forward + judgements</i>)	.366	.799
rev (<i>reverse only</i>)	.585	.799
<i>trained on BOTH directions</i>		
flip	.512	.744
mix50	.469	.732

147 **The replication fails in the direction that matters.** Our cft arm reaches 0.1% strict reverse
148 success at 7B, against a reported 39–52%. The gap is not merely a shortfall, since cft lands 12.8 pp
149 below the untouched model $[-14.8, -10.9]$, which itself scores 12.9%, and forward-only training
150 lands equally far below. Both recipes remove a capability the base model already had, and the
151 contrastive one does not restore it. We report this at 7B because that is the size at which the original
152 study’s open-source results sit. Our corpus is Python and smaller than theirs, so we claim the method
153 does not reproduce here and that the mechanism it was credited with is available for free.

154 **Where the fix does not work, and this is the main limit on “free”.** The headline 33% averages five
155 transforms that behave nothing alike, and the average conceals a hard ceiling. On the two structural
156 transforms the bidirectional arms reach about 80%. On the three renaming transforms every arm we
157 trained sits at or below 2.3%, and that includes rev, which sees nothing but deobfuscation examples
158 for the whole of training (Appendix H). The reason is not a shortage of data or capacity but the task
159 itself. Once total_price has been rewritten to v_a3f2, the original name is not present anywhere
160 in the input, so no quantity of reverse supervision can recover it, so a practitioner should not expect
161 reverse training to help on transforms that destroy information rather than rearrange it. The flip is
162 free on budget, and it works only where the transformation has an inverse left to learn.

163 A second correction concerns the measurement rather than the method. The published criterion
164 certifies very little, since outputs that pass it recover almost exactly as many original identifiers as
165 the model’s outputs do in general (Appendix K).

166 5 What bidirectional training costs, and why that cost is also misattributed

167 Calling the flip free is a claim about training budget, and we scope it that way. On forward accuracy
168 it is genuinely free, since flip matches sft on every transform (Appendix I). On general coding
169 ability it is not. We measure that with HumanEval+ [Liu et al., 2023], a benchmark of 164 standalone
170 programming problems scored by running the model’s solution against tests, where pass@1 is the
171 fraction solved on the model’s first attempt.

At 7B every arm trained on a *single* direction lands within 1.2 points of the untouched model, including `rev`, which is trained on reverse data exclusively. The two arms carrying both directions lose 6 to 7 points. The comparison that isolates this is `rev` against `mix50`. They use the same number of examples and optimizer steps, `rev` is taught *less* ($0.43\times$ the supervised tokens against $0.71\times$), and they reach the same reverse success (32.9% against 32.8%), yet `mix50` gives up 6.7 points of `pass@1` and `rev` gives up none. Data volume, compute and reverse exposure are all held level, so what is left is whether the model must learn a rewrite and its inverse at the same time. This narrows the cost claim rather than removing it. Bidirectional training costs 6 to 7 points; reverse data on its own does not.

Two cautions. The scales disagree, and at 1.5B the grouping fails, because forward-only training is there the most damaging arm, which reads as ordinary overfitting to a narrow task. And 104 of the 164 HumanEval+ problems have their reference solutions in our training corpus, since HumanEval is one of its three sources, so every trained arm saw them while the untouched model did not. Comparisons *among* trained arms, which is what the grouping rests on, are unaffected, but the 6 to 7 point figures against the untouched model are not clean.

6 Related work

Directionality and the reversal curse. The nearest literature to our phenomenon is the reversal curse. Berglund et al. [2024] show that a model trained on “A is B” does not thereby learn “B is A”, and Golovneva et al. [2024] cure much of that failure with reverse training, in which strings are additionally presented reversed, evaluated in both data-matched and compute-matched settings. Our budget-matched arm is the same manoeuvre, and their result is a reason to expect ours.

What we report is a different failure with the same symptom. The reversal curse is a failure to *generalize* to a direction never trained, so the capability was never present. Here the untouched model can already work in reverse, scoring 12.9% at 7B, and forward-only fine-tuning *removes* it, taking it to 0.0%. Destroying an ability the model had is not the same event as failing to acquire one, and it is caused by the fine-tuning a practitioner chooses to run. The loss is also selective, leaving general coding ability untouched, whereas the reversal curse describes a limit on what a training distribution can teach rather than damage to an existing skill. The inverse here is semantic too, undoing a program transformation rather than reordering tokens or entities.

Deobfuscation. Nikiema et al. [2025] is the study replicated here. A larger literature trains models to recover source from a transformed form, whether obfuscated, compiled or decompiled [Roziere et al., 2021, Tan et al., 2024, Hu et al., 2026], and our setting differs in where the reverse supervision comes from, since we re-read pairs that already exist. Guzmán Lorenzo [2026] find that misleading identifiers survive deobfuscation almost always, agreeing with our renaming ceiling. All arms are LoRA adapters [Hu et al., 2022].

7 Limitations and conclusion

Our study is Python and JavaScript rather than Java, two sizes of one model family with the 7B arms at a single seed, and five transforms rather than the original’s three, and the general-capability benchmark overlaps our training corpus. Appendix C has the full list.

Fine-tuning a model on one direction of a reversible code transformation removes the other direction, and removes it selectively, leaving general coding ability untouched at 7B while reverse success falls to zero. That is the finding a practitioner should carry away, because a directional blind spot of this kind will not show up in a general benchmark. It is also cheap to prevent, since the training data for the missing direction is the data already in hand read backwards, and an arm that replaces half its forward examples with reversed ones recovers the direction at no additional training cost.

The remedy proposed for this failure in prior work is credited to the wrong factor, since the recovery comes from the reverse data it supplies rather than from its objective, and the cost is misattributed the same way. A factor credited with an effect should be tested against the ablation its own training instances supply.

References

- Lukas Berglund, Meg Tong, Maximilian Kaufmann, Mikita Balesni, Asa Cooper Stickland, Tomek Korbak, and Owain Evans. The reversal curse: LLMs trained on “A is B” fail to learn “B is A”. In *International Conference on Learning Representations (ICLR)*, 2024.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Olga Golovneva, Zeyuan Allen-Zhu, Jason Weston, and Sainbayar Sukhbaatar. Reverse training to nurse the reversal curse. *arXiv preprint arXiv:2403.13799*, 2024.
- Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. CRUXEval: A benchmark for code reasoning, understanding and execution. *arXiv preprint arXiv:2401.03065*, 2024.
- Luis Guzmán Lorenzo. Poisoned identifiers survive LLM deobfuscation: A case study on claude opus 4.6. *arXiv preprint arXiv:2604.04289*, 2026.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Li Hu, Xiuwei Shang, Jieke Shi, Shaoyin Cheng, Junqi Zhang, Gangyang Li, Zhou Yang, Weiming Zhang, and David Lo. Can LLMs deobfuscate binary code? a systematic analysis of large language models into pseudocode deobfuscation. *arXiv preprint arXiv:2604.08083*, 2026.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Serge Lionel Nikiema, Jordan Samhi, Micheline Bénédicte Moumoula, Albérick Euraste Djiré, Abdoul Kader Kaboré, Jacques Klein, and Tegawendé F. Bissyandé. Using contrastive learning to improve two-way reasoning in large language models: The obfuscation task as a case study. *arXiv preprint arXiv:2509.05553*, 2025.
- Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. CodeBLEU: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297*, 2020.
- Baptiste Roziere, Marie-Anne Lachaux, Marc Szafraniec, and Guillaume Lample. DOBF: A deobfuscation pre-training objective for programming languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Simone Scalabrino, Mario Linares-Vásquez, Rocco Oliveto, and Denys Poshyvanyk. A comprehensive model for code readability. *Journal of Software: Evolution and Process*, 30(6), 2018.
- Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. LLM4Decompile: Decompiling binary code with large language models. In *Findings of the Association for Computational Linguistics (EMNLP Findings)*, 2024.

264 Appendix

265 A Reimplementation fidelity, deviations, and the determinism floor

266 **The original objective is instance-level.** (i) Its three terms are described only as instance pools
 267 with counts (“30 000 instances, 10 000 each for positive classification, negative classification, and
 268 obfuscation task generation”); the words InfoNCE, margin, cosine, temperature and embedding
 269 do not appear in the paper. (ii) Two of its three headline CFT numbers come from models tuned
 270 through provider fine-tuning APIs, which accept only supervised input→output text pairs, so a cus-
 271 tom embedding-space loss is not expressible there. (iii) Open-source models are tuned with plain
 272 LoRA and no modified objective. We therefore realise $\mathcal{L}_{\text{pos}} + \mathcal{L}_{\text{neg}} + \mathcal{L}_{\text{gen}}$ as joint next-token cross-
 273 entropy over the union of the three pools, which is what a sum of per-task losses over balanced
 274 datasets is.

275 **Readability is a proxy, and the choice does not matter.** Eq. (1) tests whether an output is about
 276 as readable as the original, and the original study measures that with Scalabrino et al. [2018]. We do
 277 not have that model, so R is the weighted proxy of Table 6, calibrated by measurement rather than
 278 guessed, separating clean code from fully minified code at 8% false positives and 89% detection
 279 over 400 programs. Because a substituted measure inside a reimplemented criterion is exactly the
 280 kind of deviation that can quietly carry a result, we measured its influence directly by recomputing
 281 the 7B reverse cells under four treatments of the readability conjunct.

arm	strict reverse success (%)			
	tolerance 0.1 (reported)	term removed	tolerance 0.0	tolerance 0.3
base	12.9	13.5	10.6	13.5
sft	0.0	0.0	0.0	0.0
cft	0.1	0.1	0.0	0.1
fwd2x	0.1	0.1	0.0	0.1
rev	32.9	32.9	32.1	32.9
flip	33.5	33.6	32.3	33.5
mix50	32.8	33.0	31.9	33.0

283 Removing the readability conjunct altogether changes no arm by more than 0.6 pp and changes the
 284 mix50 minus sft contrast by 0.2 pp. Across all four treatments the spread is at most 1.3 pp on any
 285 trained arm and 2.9 pp on the untuned model, against a headline effect of 33 points. The reason is
 286 visible in the criterion itself. Once an output has to parse, be textually far from the obfuscated input,
 287 and execute correctly, it is essentially always at least as readable as the original on any reasonable
 288 measure, so the conjunct is the deciding factor on only 0.2% of trials for flip and mix50 and 1.0%
 289 for base. Substituting the readability model therefore cannot account for our conclusions. The one
 290 number where the treatment has real leverage is the untuned model’s *published-criterion* rate, which
 291 ranges from 17.6% to 24.3% across the four settings, and we place no claim on that quantity.

292 **Two deliberate deviations.** Our negatives are the *obfuscated* variant with one operator mutated
 293 and are execution-verified to differ from their parent, rather than clean programs of different se-
 294 mantics; under the original construction the question “is program B obfuscated?” predicts the label
 295 perfectly, so a model can score well on the auxiliary terms without comparing semantics at all. And
 296 our neg pool is 24% short of parity (7 383 gen / 5 611 pos / 5 611 neg) because mutants must sur-
 297 vive execution verification. That shortfall cannot plausibly explain a 30-point gap: measured on the
 298 tokenized corpus, the objective’s supervised-token mass is 97.7% $\mathcal{L}_{\text{gen}} / 1.13\% \mathcal{L}_{\text{pos}} / 1.13\% \mathcal{L}_{\text{neg}}$,
 299 because a gen target is a whole program (196.7 tokens on average) while a pos/neg target is the
 300 single token YES or NO (3.0 tokens). Restoring neg to parity moves the auxiliary share from 2.26%
 301 to $\approx 2.6\%$.

302 **The cross-pass determinism floor.** Evaluating the untuned base arm in five passes over a byte-
 303 identical 300-program set gives strict reverse rates of 2.9/2.9/2.9/2.7/2.7%. The program sets are
 304 identical in all five, and the two passes carrying the same arm list are bit-identical on all 1500 gen-
 305 erations; the others differ on 6–8% of generations and flip 4–8 graded trials. The cause is therefore

batch composition rather than corpus drift or a scorer change. The inference engine batches continuously, so which arms share a pass changes reduction order and occasionally an argmax. We treat ± 0.3 pp as the resolution of any cross-pass comparison. It bounds nothing in the body, whose effects are 30 points, but it is the reason Appendix L carries an untuned anchor row.

B The reverse-data dose response

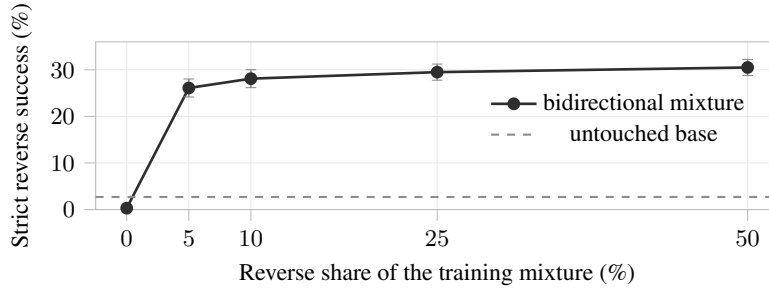


Figure 2: **A little reverse data does almost all of the work.** Strict reverse success against the reverse share of the training mixture, Qwen2.5-Coder-1.5B, 300 programs, 1 500 graded trials per point. The zero-dose point is forward-only SFT. Replacing just 5% of the forward pairs with their own reverse recovers 26.1% against 0.3% for forward-only — 85% of what a 50% share recovers. The remaining climb is real but small (+4.5 pp [+3.2, +5.8] from 5% to 50%) and has flattened by 25% (+1.1 pp [+0.0, +2.1] from 25% to 50%). Bars are 95% cluster bootstraps by program; because the design is paired, differences are quoted from the contrast table rather than read off the bars.

The curve rises steeply and then flattens. Replacing 5% of the forward pairs recovers +25.7 pp [+23.9, +27.7] over forward-only training, which is 85% of what a 50% share recovers; the remaining climb is +4.5 pp [+3.2, +5.8] and has flattened by a quarter share, where 50% against 25% is +1.1 pp [+0.0, +2.1] with an interval touching zero.

C Limitations, in full

Language and corpus. The original study is Java over 10 000 CodeNet programs, while ours is Python over 7 383 forward instances per epoch plus a JavaScript replication of the four load-bearing arms at 1.5B. Two languages is evidence that the effect is not an artifact of one, but neither is Java, so a Java-specific effect is not excluded. **Scale and family.** Two sizes of one family, Qwen2.5-Coder 1.5B and 7B. The original’s strongest CFT numbers come from commercial models we cannot fine-tune the same way, though our 7B result is at the tier of its open-source panel, and the 7B arms are single-seed. **The general-capability probe overlaps the corpus,** as g5 sets out; a held-out probe such as MBPP would settle it and we did not run one.

Transform coverage. String encryption, one of the original’s three transforms and its hardest, is quarantined in our corpus as a held-out family for a separate study and is never trained on, so we report five conditions where the original reports three.

Auxiliary pool ratio. Our neg pool is $0.76\times$ the size of the others because negatives must survive execution verification. The token-share arithmetic in Appendix A bounds the auxiliary signal at $\approx 2.6\%$ of supervised tokens even at parity, so the shortfall cannot plausibly account for a 30-point gap, but we did not run the parity sweep.

Pairing on HumanEval+. The cost table is a single greedy pass per arm and only aggregate counts were retained for the cells run before this draft, so g5’s grouping is a pattern across the arms rather than a set of per-pair significance tests.

334 D General-capability cost of the 1.5B dose ladder

arm	reverse share	HumanEval+ plus
sft	0%	.329
mix5	5%	.537
mix10	10%	.555
mix25	25%	.494
mix50	50%	.469
rev	100%	.585
base	n/a	.646

336 Qwen2.5-Coder-1.5B, 164 tasks, 0 scorer errors. The retained capability is *not* monotone in reverse
 337 share. `mix5` and `mix10` differ by 3 of 164 tasks, and `rev`, which has the largest reverse share, retains
 338 the most. We therefore draw no dose-response conclusion on this axis at 1.5B. The 7B grouping in
 339 §5 does not rest on these rows.

340 E Measured training budget (7B)

Table 5: Measured per-epoch training budget, 7B Python corpus, relative to forward-only SFT. Supervised tokens are the tokens the loss is actually computed on.

arm	instances	supervised tokens	sequence tokens ($\propto$ FLOPs)	steps
fwd (<i>reference</i>)	1.00×	1.00×	1.00×	1.00×
fwd2x	1.00×	1.00×	1.00×	2.00×
mix50	1.00×	0.71×	1.05×	1.00×
rev	1.00×	0.43×	1.09×	1.00×
flip	2.00×	1.43×	2.09×	2.00×
cft	2.52×	1.02×	2.65×	2.52×
cftflip	3.52×	1.45×	3.74×	3.52×

341 F Scoring quantities

Table 6: The quantities used to score a trial. *Executes* is checked by running the program on the test inputs that ship with it. *R* returns 0 for a non-parsing output, since the readability of a non-program is undefined and 0 is the only resolution that cannot manufacture a success.

symbol	range	meaning
$\text{parses}(\hat{y})$	$\{0, 1\}$	the output is non-empty and parses in the target language
$S(a, b)$	$[0, 1]$	CodeBLEU similarity between two programs [Ren et al., 2020]
$R(a)$	$[0, 1]$	readability, a weighted sum of identifier meaningfulness (0.55), identifier length, line length and nesting depth (0.15 each)
$E(\hat{y})$	$\{0, 1\}$	the output executes and reproduces x ’s outputs on every test input

Table 7: The five obfuscating transforms. Each is applied on its own to the original function. We keep the short codes because the appendix tables report results per transform.

code	family	what it does
L1b	identifier	renames identifiers to misleading names, so <code>count</code> might become <code>buffer</code>
L1r	identifier	renames identifiers to random strings such as <code>v_a3f2</code>
L2	identifier	renames identifiers to single letters and strips type annotations
S1	structural	flattens control flow into a single dispatch loop driven by a state variable
S2	structural	inserts always-true tests and unreachable code

G The five obfuscating transforms

H Strict reverse success by condition (7B)

arm	L1b	L1r	L2	S1	S2
base	.023	.110	.103	.330	.077
sft	.000	.000	.000	.000	.000
cft	.000	.000	.000	.003	.000
fwd2x	.000	.000	.000	.003	.000
rev	.003	.010	.017	.817	.797
flip	.017	.013	.023	.827	.797
mix50	.010	.007	.023	.803	.797

300 programs per cell, simple strategy, e2_budget_qwen7b. The capability is almost entirely structural, since the identifier conditions are at or below 2.3% for every arm, including reverse-only training, because adversarial renaming destroys information that no amount of reverse supervision can restore.

I Forward accuracy by condition (7B)

arm	L1b	L1r	L2	S1	S2
base	.950	.960	.963	.933	.837
sft	.970	.950	1.000	.933	1.000
cft	.973	.960	.997	.947	1.000
fwd2x	.967	.960	1.000	.917	1.000
rev	.940	.950	.957	.970	.820
flip	.970	.960	1.000	.927	.997
mix50	.973	.957	.990	.883	1.000

Execution-verified forward success. `flip` matches `sft` everywhere; bidirectionality is free in this direction too.

J Reverse prompting strategies (7B)

system	simple	few_shot	cot	augmented	range
base	13.2 [11.3, 15.3]	18.1 [15.7, 20.7]	21.9 [19.4, 24.5]	21.4 [18.8, 24.1]	8.7
sft	0.0 [0.0, 0.0]	1.6 [1.0, 2.3]	5.3 [4.2, 6.3]	5.1 [4.1, 6.1]	5.3
cft	0.1 [0.0, 0.2]	0.5 [0.1, 0.9]	1.5 [0.9, 2.1]	1.2 [0.7, 1.7]	1.4
rev	32.9 [31.5, 34.5]	32.5 [31.0, 34.0]	32.5 [31.0, 34.2]	32.9 [31.3, 34.4]	0.4
flip	33.7 [32.1, 35.3]	32.6 [31.1, 34.1]	32.8 [31.2, 34.5]	32.6 [31.1, 34.1]	1.1
mix50	32.8 [31.3, 34.4]	31.3 [29.6, 32.9]	30.3 [28.9, 31.9]	31.7 [30.0, 33.3]	2.5

355 Strict reverse success (%), cluster-bootstrap CIs by `program_id`; all six arms scored in one eval-
 356 uation pass (`e7_strategies_qwen7b`), best strategy per arm in bold. Both forward-only arms at
 357 their best strategy fall below the untuned base at its worst. The bidirectional arms (lower block) are
 358 nearly flat across strategies while the untuned base spans 8.7 points, so capability installed by data
 359 does not depend on how it is elicited.

360 K What the reverse criterion certifies (7B, simple)

system	cond	paper %	strict %	id-rec (passes)	id-rec (all)	lift	floor
base	L1b	4.7	2.3	0.326	0.363	-0.037	0.354
base	L1r	19.0	11.0	0.428	0.423	+0.005	0.349
base	L2	18.7	10.3	0.437	0.420	+0.017	0.389
base	S1	64.7	33.0	0.895	0.906	-0.011	1.000
base	S2	9.3	7.7	0.750	0.878	-0.127	1.000
flip	S1	90.7	82.7	0.999	0.999	+0.001	1.000
flip	S2	79.7	79.7	1.000	0.999	+0.000	1.000
mix50	S1	90.3	80.3	0.994	0.994	-0.000	1.000
mix50	S2	79.7	79.7	0.999	0.999	-0.000	1.000

362 *floor* is the identifier recall of the obfuscated input itself, which is what echoing scores. *lift* is id-
 363 recall among criterion-passing outputs minus id-recall among all outputs; a lift near zero means the
 364 criterion certifies nothing about identifier recovery. The structural conditions have a floor of exactly
 365 1.000, so identifier recall does not discriminate there at all.

366 L Seed replication (1.5B, strict reverse success %)

arm	seed 17	seed 42	Δ
sft	0.4	0.4	+0.0
cft	0.3	0.4	+0.1
flip	31.5	30.9	-0.6
rev	31.5	31.2	-0.3
mix50	30.6	30.0	-0.6
flipsym	30.5	30.8	+0.3
base (untuned anchor)	2.9	2.7	-0.2

368 Identical harness, program set, criteria and decoding; the adapter seed is the only difference.
 369 Columns come from two evaluation passes (`e1_qwen1.5b`, `e2_seeds_qwen1.5b`). The base row
 370 has no seed and is the same untouched model in both, so it calibrates the table. Roughly 0.2 pp of
 371 the movement in every other row is cross-pass noise rather than seed variance (§2). The two arms
 372 carrying the paper’s null move by ≤ 0.1 . `flipsym` trains `flip`’s mixture under a single shared sys-
 373 tem prompt and is a control against the two directions being separable merely because they arrived
 374 under two personas.

375 M JavaScript replication (1.5B, strict reverse success %)

arm	JavaScript	contrast vs. Python
base	3.8 [2.5, 5.3]	(Python 2.9)
sft	0.0 [0.0, 0.0]	sft-base -3.8 [-5.6, -2.4]
cft	0.4 [0.0, 0.8]	cft-sft +0.4 [+0.0, +0.8]
mix50	30.3 [27.4, 33.1]	mix50-sft +30.3 [+28.3, +32.5]
flip	30.4 [27.5, 33.2]	flip-mix50 +0.1 [-0.2, +0.5]

377 167 programs, 835 graded trials per arm, same conditions and criteria as the Python corpus. Every
 378 qualitative conclusion holds. Forward-only tuning lands below the untouched model, the contrastive
 379 objective does not repair it, and the budget-matched bidirectional arm captures the whole effect.

380 Here it is statistically indistinguishable from the doubled-data arm, whereas in Python the latter led
381 by +0.7 pp.